ICPC Team Reference

CoogLimitExceeded

Contents

Bitset Funcs **2**

Dijkstra **3**

Dir Template **4**

Egcd **5**

Floyd Warshall **6**

Modinv Fastpower And Mod Comb **8**

Multiset Funcs **9**

Segment Tree **10**

Sieve Primes **12**

Smallest Prime Factor **13**

Template Code **14**

Trie **15**

Vector Funcs **16**

Bitset Funcs

```cpp
// bit_utils.hpp
#pragma once
#include <bits/stdc++.h>
using namespace std;
#define ll long long

// Convert string of bits ("1011") to long long
ll bitset_to_ll(const string &s) {
    return (ll)bitset<64>(s).to_ullong();
}

// Convert long long to bitset string of length n
string ll_to_bitset(ll x, int n = 64) {
    bitset<64> b(x);
    string s = b.to_string();
    return s.substr(64 - n, n);
}

// Count number of set bits
int count_bits(ll x) {
    return (int)bitset<64>(x).count();
}

// Check if k-th bit (0-indexed from right) is set
bool get_bit(ll x, int k) {
    return (x >> k) & 1LL;
}

// Set k-th bit
ll set_bit(ll x, int k) {
    return x | (1LL << k);
}

// Clear k-th bit
ll clear_bit(ll x, int k) {
    return x & ~(1LL << k);
}

// Toggle k-th bit
ll toggle_bit(ll x, int k) {
    return x ^ (1LL << k);
}

// Get lowest set bit position (1-indexed). Returns 0 if none.
int lowest_set_bit_pos(ll x) {
    if (x == 0) return 0;
    return __builtin_ctzll(x) + 1;
}

// Get highest set bit position (1-indexed). Returns 0 if none.
int highest_set_bit_pos(ll x) {
    if (x == 0) return 0;
    return 64 - __builtin_clzll(x);
}
```

Dijkstra

```cpp
#include <bits/stdc++.h>
using namespace std;
#define ll long long

struct edge {
    int node; ll weight; //node on the end of edge, weight of edge
};
// to customize the priority queue, we can use a custom comparator
bool operator < (edge a, edge b){
    return a.weight > b.weight; // O(1)
}
const int N = 1e5;
ll distance_to_node[N];
int parent[N];
vector<edge> graph[N];

void dijkstra(int start, int destination)
{
    memset(distance_to_node,63,sizeof distance_to_node); // O(N)
    priority_queue<edge>pq; // O(1)
    pq.push({start,0}); // O(log 1) = O(1)
    distance_to_node[start] = 0; parent[start] = -1; // O(1)

    while(!pq.empty()){ // loop runs up to O((V + E) log V) overall
        auto current = pq.top(); pq.pop(); // O(log V)

        if (current.node == destination) break; // O(1)

        for (auto e : graph[current.node]){ // iterates over neighbors -> O(E) total
    across all loops
            ll new_weight = current.weight + e.weight; // O(1)
            if (distance_to_node[e.node] > new_weight) { // O(1)
                distance_to_node[e.node] = new_weight; // O(1)
                pq.push({e.node, new_weight}); // O(log V)
                parent[e.node] = current.node; // O(1)
            }
        }
    }
  //no path
    if (parent[destination] == 0)
      return; // O(1)

    vector<int> path; // O(1)
    int cur = destination; // O(1)
    while(cur != -1){ // O(V) in worst case
        path.push_back(cur); // O(1) amortized
        cur = parent[cur]; // O(1)
    }
    reverse(path.begin(), path.end()); // O(V)
}
```

Dir Template

```cpp
#include <bits/stdc++.h>
using namespace std;

// TODO: Implement your algorithm/data structure here
// This file should contain ONLY the algorithm/data structure code, NO main() function
//
// Example:
// struct my_data_structure {
//     // implementation
// };
//
// or
//
// void my_algorithm(vector<int>& arr) {
//     // implementation
// }
```

Egcd

```cpp
#include <bits/stdc++.h>
using namespace std;

#define ll long long

// Extended Euclidean Algorithm
// Returns gcd(a, b) and finds x, y such that a*x + b*y = gcd(a, b)
ll egcd(ll a, ll b, ll &x, ll &y) {
    if (b == 0) {          // Base case
        x = 1; y = 0;
        return a;
    }
    ll x1, y1;
    ll g = egcd(b, a % b, x1, y1);  // Recurse
    x = y1;                         // Update x
    y = x1 - (a / b) * y1;          // Update y
    return g;                       // Return gcd
}
// Time complexity: O(log(max(a, b)))
```

Floyd Warshall

```cpp
#include <bits/stdc++.h>
using namespace std;

// --- Your "edge" struct (assuming you have this) ---
struct edge {
    int from, to, distance;
};

// --- ICPC-Style Floyd-Warshall Library Code ---

// !! Change this based on problem constraints !!
const int MAXN = 103; // Max number of nodes + 1 (for 1-indexing)
using ll = long long;
const ll INF = 1e18;

struct FloydWarshall {
    ll dp[MAXN][MAXN];
    int next_hop[MAXN][MAXN];
    int N; // Actual number of nodes

    // Initialize for 'n' nodes (1-indexed)
    FloydWarshall(int nodes) {
        N = nodes;
        for (int i = 1; i <= N; i++) {
            for (int j = 1; j <= N; j++) {
                dp[i][j] = (i == j) ? 0 : INF;
                next_hop[i][j] = -1; // -1 means no path
            }
            next_hop[i][i] = i;
        }
    }

    // Add a directed edge
    void add_edge(int u, int v, ll w) {
        if (w < dp[u][v]) { // Handle parallel edges
            dp[u][v] = w;
            next_hop[u][v] = v; // The first hop from u to v is v
        }
    }

    // Run the O(N^3) algorithm
    void run() {
        for (int k = 1; k <= N; k++) {
            for (int i = 1; i <= N; i++) {
                for (int j = 1; j <= N; j++) {
                    // Check for INF to prevent overflow
                    if (dp[i][k] == INF || dp[k][j] == INF) continue;

                    ll new_dist = dp[i][k] + dp[k][j];
                    if (new_dist < dp[i][j]) {
                        dp[i][j] = new_dist;
                        // The first hop from i->j is the same as the first hop from i->
    k
                        next_hop[i][j] = next_hop[i][k];
                    }
                }
            }
        }
    }

    // Get the shortest path distance from u to v
    ll get_dist(int u, int v) {
        return dp[u][v]; // Will be INF if no path
    }

    // Reconstruct the path from u to v
    vector<int> get_path(int u, int v) {
        if (dp[u][v] == INF) {
            return {}; // No path
        }

```

```
        vector<int> path;
        int curr = u;
        while (curr != v) {
            if (curr == -1) return {}; // Should not happen if dp[u][v] != INF
            path.push_back(curr);
            curr = next_hop[curr][v];
        }
        path.push_back(v);
        return path;
    }

    // Check for negative cycles
    bool has_negative_cycle() {
        for (int i = 1; i <= N; i++) {
            if (dp[i][i] < 0) {
                return true;
            }
        }
        return false;
    }
};
```

Modinv Fastpower And Mod Comb

```cpp
#include <bits/stdc++.h>
using namespace std;

#define ll long long

ll mod = 1e9 + 7;
ll fact[2009], inv[2009];

// Fast exponentiation: b^e mod m
ll fastPower(ll b, ll e, ll m) {
    if (e == 0) return 1;
    ll hp = fastPower(b, e / 2, m);
    hp = (hp * hp) % m;
    if (e % 2) hp = (hp * b) % m;
    return hp % m;
}

// Modular inverse using Fermat's little theorem
ll modularInverse(ll b, ll m) {
    return fastPower(b, m - 2, m);
}

// Precompute factorials and inverses up to mx
void factorials(int mx) {
    fact[0] = inv[0] = 1;
    for (int i = 1; i < mx; i++) {
        fact[i] = (i * fact[i - 1]) % mod;
        inv[i] = modularInverse(fact[i], mod);
    }
}

// Compute nCr mod m
ll nCr(ll n, ll r, ll m) {
    if (r < 0 || r > n) return 0;
    ll a = fact[n];
    ll b = (inv[n - r] * inv[r]) % m;
    return (a * b) % m;
}
```

Multiset Funcs

```cpp
#include <bits/stdc++.h>
using namespace std;

// TODO: Implement your algorithm/data structure here
// This file should contain ONLY the algorithm/data structure code, NO main() function
//
// Example:
// struct my_data_structure {
//     // implementation
// };
//
// or
//
// void my_algorithm(vector<int>& arr) {
//     // implementation
// }
```

Segment Tree

```cpp
// templated node type.
template <typename T>
struct node
{
    T val;
};

// generic segment tree storing nodes of type node<T>
template <typename T>
struct segment_tree
{
    int size = 1;
    vector<node<T>> nodes;

    // for safe margin and no loop. we can initialize by 4*num_leaves
    // NOT RECOMMENDED for multiple initialization or large N
    void init(int num_leaves)
    {
        while (size < num_leaves)
            size <<= 1;
        nodes.resize(size << 1);
        nodes.assign(size << 1, neutral_node());
    }

    node<T> neutral_node() const
    {
        return {T{}}; // like 0 or empty string
    }

    node<T> single(T val) const
    {
        return {val};
    }

    // merge operation - here sum for generic T that supports +, it can be product, min, max, etc.
    node<T> merge(const node<T> &node1, const node<T> &node2) const
    {
        return single(node1.val + node2.val);
    }

    void build(vector<T> &v, int x, int lx, int rx)
    {
        if (rx - lx == 1)
        {
            if (lx < (int)v.size())
            {
                nodes[x] = single(v[lx]);
            }
            return;
        }

        int m = (rx + lx) >> 1;
        build(v, 2 * x + 1, lx, m);
        build(v, 2 * x + 2, m, rx);
        nodes[x] = merge(nodes[2 * x + 1], nodes[2 * x + 2]);
    }

    void build(vector<T> &v)
    {
        build(v, 0, 0, size);
    }

    // desired index, value (as node), current node index, left and right bounds of node
    void set_index_to_value(int i, const node<T> &v, int x, int lx, int rx)
    {
        if (rx - lx == 1)
        { // we are at desired node
            nodes[x] = v;
            return;
        }
```

```
71
72          int m = (rx + lx) >> 1;
73          if (i < m)
74          { // go left
75              set_index_to_value(i, v, 2 * x + 1, lx, m);
76          }
77          else
78          { // go right
79              set_index_to_value(i, v, 2 * x + 2, m, rx);
80          }
81          nodes[x] = merge(nodes[2 * x + 1], nodes[2 * x + 2]);
82      }
83
84      void set_index_to_value(int i, T val)
85      {
86          set_index_to_value(i, single(val), 0, 0, size);
87      }
88
89      node<T> get(int l, int r, int x, int lx, int rx) const
90      {
91          if (l >= rx || lx >= r)
92          { // out of bounds of query
93              return neutral_node();
94          }
95          if (lx >= l && rx <= r)
96          {
97              return nodes[x]; // all within bounds, take this node
98          }
99
100         int m = (rx + lx) >> 1;
101         node<T> lt = get(l, r, 2 * x + 1, lx, m);
102         node<T> rt = get(l, r, 2 * x + 2, m, rx);
103         return merge(lt, rt);
104     }
105
106     node<T> get(int l, int r) const
107     {
108         return get(l, r, 0, 0, size);
109     }
110 };
```

Sieve Primes

```cpp
#include <bits/stdc++.h>
using namespace std;

const int N = 1e5;
bool prime[N];

// Sieve of Eratosthenes: marks prime[i] = true if i is prime
// Time: O(N log log N), Space: O(N)
void sieve(int X) {
    memset(prime, true, sizeof prime);
    prime[0] = prime[1] = false;
    for (int i = 2; i * i <= X; i++) {
        if (prime[i]) {
            for (int j = i * i; j <= X; j += i) prime[j] = false;
        }
    }
}
```

Smallest Prime Factor

```cpp
#include <bits/stdc++.h>
using namespace std;

const int N = 1e7;        // Maximum number for preprocessing

int spf[N] = {};           // Smallest prime factor array

// Preprocess smallest prime factors for all numbers up to N
void preprocess() {
    spf[0] = spf[1] = 0;          // 0 and 1 are not prime
    for (int i = 2; i < N; ++i) spf[i] = i;  // Initialize spf[i] = i

    for (int i = 2; i * i < N; ++i) {
        if (spf[i] == i) {            // i is prime
            for (int j = i * i; j < N; j += i)
                if (spf[j] > i) spf[j] = i;  // Update smallest prime factor
        }
    }
}

// Time complexity: O(N log log N) using sieve
// Space complexity: O(N)

/* USAGE:

int x = 60;
vector<int> factors;
while (x > 1) {
    factors.push_back(spf[x]); // get smallest prime factor
    x /= spf[x];               // reduce x
}
*/
```

Template Code

```cpp
#include <bits/stdc++.h>
#include <ext/pb_ds/assoc_container.hpp>
#include <ext/pb_ds/tree_policy.hpp>

using namespace std;
using namespace __gnu_pbds;
template<typename T> using ordered_set = tree<T, null_type, less<T>, rb_tree_tag,
    tree_order_statistics_node_update>;

typedef long long ll;

typedef vector<int> vi;
typedef vector<ll> vll;

#define el '\n'
#define pb push_back

#define all(x) (x).begin(),(x).end()
#define rall(x) (x).rbegin(),(x).rend()

#define OO (int)2e9
#define LOO (ll)9e18

#define bitCountLL(x) __builtin_popcountll(x)
#define bitCount(x) __builtin_popcount(x)


int di[] = {-1, 1, 0, 0, 1, -1, 1, -1};
int dj[] = {0, 0, 1, -1, 1, -1, -1, 1};


const int mod = (int) 1e9 + 7;
const int N = 2e5 + 5, M = 20 + 5;

bool MULTIPLE_TEST_CASES = true;

void run_case(int case_index){
    cout << "Hello world" << endl;
}


int main() {
    // FILE IO
    freopen("input.txt", "r", stdin);
    freopen("output.txt", "w", stdout);

    // FAST IO
    ios_base::sync_with_stdio(false);
    cin.tie(nullptr);
    cout.tie(nullptr);


    int test_cases = 1;
    if (MULTIPLE_TEST_CASES) cin >> test_cases;

    for (int i = 1; i <= test_cases; ++i) {
        run_case(i);
    }
}
```

Trie

```
#include <bits/stdc++.h>
using namespace std;


//TRIE

struct Node {
    Node *next_node[26] = {};
    int freq[26] = {}; //to count frequencies of some prefix
    Node() {
        memset(next_node, 0, sizeof next_node);
        memset(freq, 0, sizeof freq);
    }
};

Node *root;

void insert(string s) {
    Node *ptr = root;
    for (int i = 0; i < (int)s.size(); i++) {
        int c = s[i] - 'a';
        if (!ptr->next_node[c])
            ptr->next_node[c] = new Node();
        ptr->freq[c]++;
        ptr = ptr->next_node[c];
    }
}

//count frequency of given prefix
int count(string s) {
    Node *ptr = root;
    for (int i = 0; i < (int)s.size(); i++) {
        int c = s[i] - 'a';
        if (!ptr->next_node[c])
            return 0;
        if (i == (int)s.size() - 1)
            return ptr->freq[c];
        ptr = ptr->next_node[c];
    }
    return 0;
}

/*
 USAGE
 root = new Node();
// insert strings, count them if 0 it DNE

  */
```

Vector Funcs

```cpp
#include <bits/stdc++.h>
using namespace std;

#define all(x) x.begin(),x.end()

// print vector neatly
template <typename T>
void print_vec(const vector<T>& v, string name = "") {
    if (!name.empty()) cout << name << " = ";
    cout << "[ ";
    for (auto& x : v) cout << x << " ";
    cout << "]\n";
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    vector<int> base = {5, 1, 3, 9, 7, 3};

    print_vec(base, "base");

    // accumulate (sum) and partial_sum (prefix sums)
    {
        vector<int> v = base;
        cout << "sum = " << accumulate(all(v), 0) << "\n";
        vector<int> pref(v.size());
        partial_sum(all(v), pref.begin());
        print_vec(pref, "partial (prefix) sum");
    }

    // count, find, any_of / all_of / none_of (each is O(n) time, O(1) space)
    {
        vector<int> v = base;
        cout << "count(3) = " << count(all(v), 3) << "\n";
        cout << boolalpha; // makes booleans printed as 'true' and 'false'
        cout << "any_of(>8): " << any_of(all(v), [](int x){return x > 8;}) << "\n";
        cout << "all_of(<10): " << all_of(all(v), [](int x){return x < 10;}) << "\n";
        cout << "none_of(<0): " << none_of(all(v), [](int x){return x < 0;}) << "\n";
        cout << noboolalpha; // disables boolalpha
    }

    // lower_bound / upper_bound (binary search helpers)
    {
        vector<int> v = base;
        sort(all(v));
        int x = 3;
        auto lb = lower_bound(all(v), x);
        auto ub = upper_bound(all(v), x);
        cout << "lower_bound(3) index = " << (lb - v.begin())
             << ", upper_bound(3) index = " << (ub - v.begin()) << "\n";
    }

    // unique + erase (deduplicate sorted vector)
    {
        vector<int> v = base;
        sort(all(v));
        // O(n log n) time, O(1) space
        v.erase(unique(all(v)), v.end());
        // O(n) time, O(1) space
        print_vec(v, "unique");
    }

    // next_permutation / prev_permutation
    {
        vector<int> v = {1, 2, 3};
        next_permutation(all(v));
        // O(n) time, O(1) space
        print_vec(v, "next_permutation");
        prev_permutation(all(v));
        // O(n) time, O(1) space
```

```
72          print_vec(v, "prev_permutation");
73      }
74
75      // nth_element (find k-th order statistic without sorting)
76      //  more than ONE query, sort instead
77      {
78          vector<int> v = base;
79          int k = 2;
80          nth_element(v.begin(), v.begin() + k, v.end());
81          // O(n) avg, O(n^2) worst (extremely rare), O(1) space
82          cout << "3rd smallest = " << v[k] << "\n";
83      }
84
85      // sort with custom comparator (e.g., by last digit)
86      {
87          vector<int> v = base;
88          sort(all(v), [](int a, int b) {
89              if (a % 10 == b % 10) return a < b;
90              return (a % 10) < (b % 10);
91          });
92          // O(n log n) time, O(1) space
93          print_vec(v, "custom sort (by last digit)");
94      }
95
96      // set operations on sorted vectors
97      {
98          vector<int> a = {1, 2, 3, 4, 5};
99          vector<int> b = {3, 4, 5, 6, 7};
100         vector<int> res;
101
102         set_union(a.begin(), a.end(), b.begin(), b.end(), back_inserter(res));
103         // O(n + m) time, O(n + m) space
104         print_vec(res, "set_union");
105
106         res.clear();
107         set_intersection(a.begin(), a.end(), b.begin(), b.end(), back_inserter(res));
108         // O(n + m) time, O(min(n, m)) space
109         print_vec(res, "set_intersection");
110
111         res.clear();
112         set_difference(a.begin(), a.end(), b.begin(), b.end(), back_inserter(res));
113         // O(n + m) time, O(n) space
114         print_vec(res, "set_difference");
115     }
116
117     return 0;
118 }
```